Unit 2
Computational Thinking & Algorithms

Correctness · clarity · Big-O · linear and non-linear structures

TRACE TABLES | $O(1)$ $O(\log n)$ $O(n)$ $O(n^2)$ | STACK LIFO | QUEUE FIFO | TREE & GRAPH

Class XI invented algorithms. Class XII asks: is it right, is it clear, and what does it cost?

1. Correctness — prove it by hand before you trust it

An algorithm is **correct** when every valid input produces the specified output and the process stops. Two classroom tools:

TRACE TABLE
A dry-run grid: line number, each variable, condition true/false, input/output. Best for loops and arithmetic slips. High effort for large n, so use a tiny example (soldier heights between 1.57 and $1.91\,\text{m}$).

STEPWISE REASONING
Argue the invariant: "after k steps, S holds the sum of the first k integers." Catches structural bugs (forgot $N = 0$) that a short trace might miss.

Table 1. Trace of "sum $1..N$" for $N = 3$

Line	i	S	Condition	Notes
init	—	0	—	$S \leftarrow 0$
loop	1	1	$i \le 3$ T	$S \leftarrow S + i$
loop	2	3	$i \le 3$ T	
loop	3	6	$i \le 3$ T	
exit	4	6	$i \le 3$ F	output 6

$$S = \frac{N(N+1)}{2} \quad \text{is the closed form you can use to check the trace.}$$

2. Clarity — modularity and readability

Modularity: one function, one job (Single Responsibility), loose coupling, abstraction. `CalculateSum()` and `FindAverage()` beat a 40-line blob.

Readability: `velocity = distance / time` documents itself. Consistent names, shallow nesting, indentation, comments only where the why is not obvious.

Refine for clarity: rename, extract a module, flatten a condition. **Refine for efficiency:** fewer steps, stop early (short-circuit), cache a repeated calculation, pick a better structure.

3. Efficiency — counting the work

Cost has two axes: **time** (steps as n grows) and **space** (extra memory). Drivers: number of steps, number of conditions, number of repetitions. Ask: does it scale? where is the bottleneck? is the cost worth the gain?

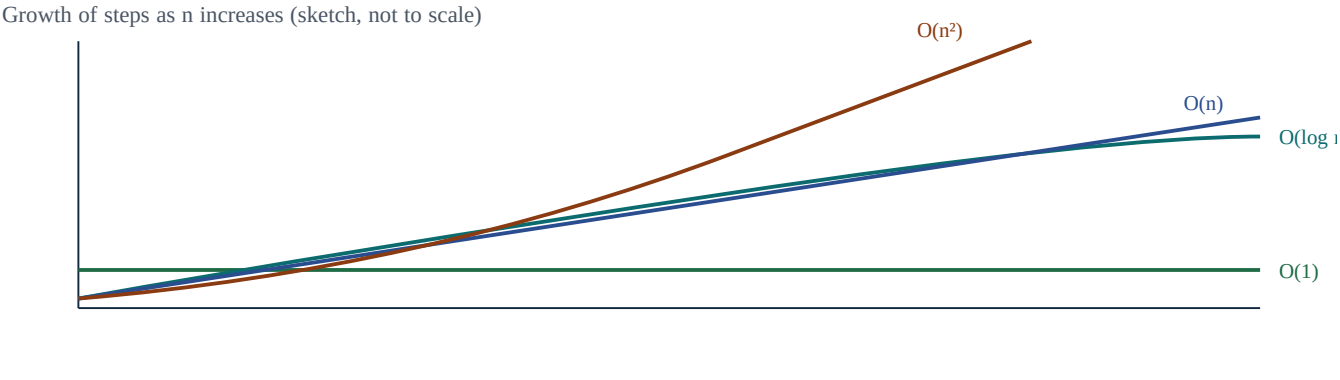


Figure 1. Big-O families the course expects you to recognise, not derive formally.

Table 2. Conceptual Big-O

Class	Meaning	Classroom example
$O(1)$	Constant — does not grow with n	Array index $A[i]$; hash lookup
$O(\log n)$	Halves the work	Binary search
$O(n)$	One pass	Linear search, “process each item once”
$O(n \log n)$	Divide-and-conquer sort (mentioned)	Efficient general sorting
$O(n^2)$	Nested loops	Bubble / selection, all-pairs compare

Repetition hierarchy: logarithmic (excellent) $\rightarrow$ linear (good) $\rightarrow$ quadratic (poor for large n). Short-circuit a compound test when the first clause already decides it.

4. Data structures — the furniture the algorithm sits on

A **data structure** is an organised way to store and retrieve values. Choose it from the *problem’s verbs*, not from fashion.

4.1 Linear structures

ARRAY
Fixed sequence, index $0..n-1$, access $O(1)$. Painful insert/delete in the middle. Classroom chairs with painted numbers.

LINKED LIST
Nodes of (data + pointer). Walk from the head. Easy insert once you are there; no jumping. Browser back/forward history.

STACK · LIFO
PUSH adds on top; **POP** removes from top. Undo, plate pile, function-call return.

QUEUE · FIFO
Enqueue at rear; **Dequeue** from front; both $O(1)$ with two pointers. Printer line, ticket window.

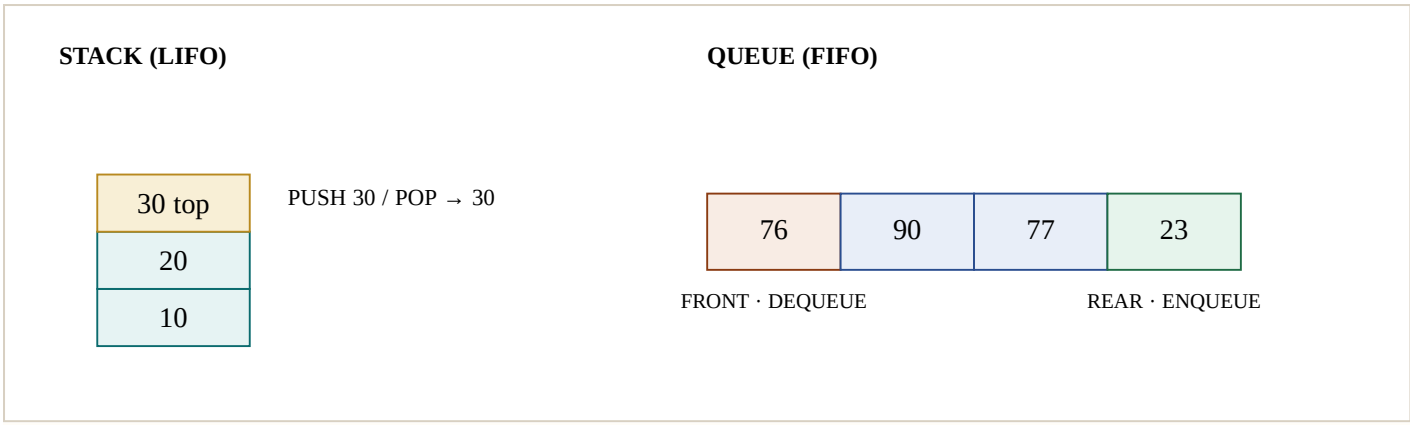
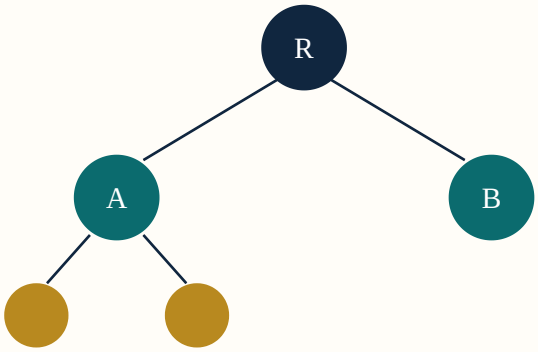


Figure 2. Stack grows upward from a single end. Queue walks from front to rear. Do not mix the verbs.

4.2 Non-linear structures

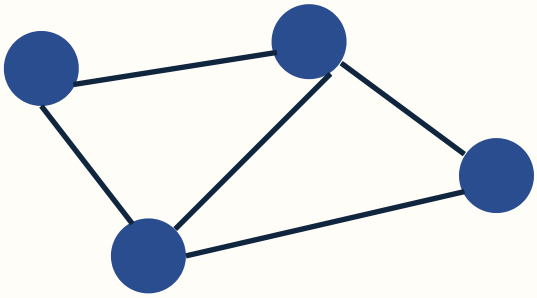
Tree

A hierarchy from one **root**. Each child has one parent. Family tree, organisation chart, folder tree.



Graph

Vertices (nodes) plus **edges** (links). No forced hierarchy. Social network, road map (cities = nodes, roads = edges). Many routes may exist.



4.3 Shared operations and retrieval traces

Insert, delete, traverse, search, sort — every structure supports some of these with different costs.

- **Array retrieval:** jump to an index, or scan forward/back.
- **List retrieval:** start at the head; walk pointers by position or by key. There is no $A[i]$ jump.
- **Queue retrieval:** the only honest removal is from the **front**.

Table 3. Pick the furniture from the verb

Problem verb	Structure
Next / previous, grades in a term	Array or list
Undo / backtrack	Stack
First-come first-served (printer)	Queue
Parent / child (org chart)	Tree
Many-to-many links (GPS, friends)	Graph

Self-check

1. Why is binary search $O(\log n)$ while scanning an unsorted list is $O(n)$?
2. PUSH 10, PUSH 20, POP, PUSH 30. What does a second POP return?
3. A college wants the fairest print spooler. Stack or queue?

Answers. (1) Each binary probe discards half; a scan may look at every cell. (2) 30 (the stack is $[10, 30]$ after the first pop-and-push). (3) Queue — FIFO is fairness.

Recap

1. Correctness: trace table + stepwise reasoning + edge cases.
2. Clarity: modules, names, shallow control flow.
3. Efficiency: $O(1), O(\log n), O(n), O(n^2)$; watch nested loops.
4. Array $O(1)$ index; list walks; stack LIFO; queue FIFO $O(1)$ enqueue/dequeue.
5. Trees branch from a root; graphs connect freely. Match structure to the problem's verb.

Act a stack with books and a queue with a classroom line.